

OPERATION NEXAAI

CEO Team Handbook

Feel free to mail me at s25089@students.iitmandi.ac.in(YASHS TIWARI)

Your Role

You are the coordination layer. You do not need to know Finance, Legal, HR, Marketing, or Engineering in depth. You need to understand what each department outputs and design a system that combines those outputs into one coherent organisational decision.

Five people on your team — one seconded from each department. They bring domain knowledge. You build the wiring.

1. What You Build

Your team owns five components. These are the four required CEO components plus the main orchestration script. All other departments build agents and tools — you wire everything together.

main.py — The Orchestration Script

Imports all 5 department agents, runs them in the correct order, passes cross-department outputs between them, assembles the submission dict, and calls submit(). This is the only file that creates a submission. Only the CEO team runs this.

converter/value_converter.py — Cross-Domain Value Converter

Finance speaks EUR. Legal speaks probability $\times$ fine. Marketing speaks market share points. Engineering speaks feasibility 0-1. HR speaks weeks. These cannot be compared directly. Your converter translates all five to one unit. The formula is your team's intellectual contribution — design it in Board Meeting 1 with department liaisons.

consensus/engine.py — Consensus Engine

A deterministic algorithm that takes all 5 calibrated, converted department outputs and produces one CEO recommendation. The same inputs must produce the same output every single time — the evaluation engine tests this by running your system 3 times. The LLM may explain the decision. The LLM cannot make the decision.

state/commitment_ledger.py — Commitment Ledger

Tracks every CEO decision across rounds. When the Deutsche Bank contract is accepted in Round 2, it is recorded. When Hurdle 3 makes that contract's conditions impossible, the ledger detects the contradiction automatically. Without this, your system will make contradictory decisions it cannot see.

state/calibration_tracker.py — Calibration Tracker

After each round, the instructor pushes a prediction accuracy report to the Global RAG. Your tracker reads it, records each department's prediction error, and updates the weights used in your Consensus Engine. A department that underestimates risk gets down-weighted. Legal underestimation should be penalised more than overestimation — your team decides by how much.

Critical: Cross-Department Data Flow

Engineering's feasibility tool **MUST** receive HR's effective FTE as an input — not a self-assumed number.

Finance's burn tool **MUST** receive HR's hiring cost and Engineering's feasibility probability as inputs. This is not optional. The evaluation engine checks cross_dept_checks in every submission.

HR → Engineering (effective_fte_at_60d)

HR → Finance (cost_per_hire)

Engineering → Finance (feasibility_probability)

3. How to Run Everything

Step 1 — Open your code editor

Open your team's code-server URL in your laptop browser. All 5 CEO team members can have it open simultaneously — each person owns specific files.

Step 2 — Activate the environment

```
# In the code-server terminal (bottom panel)
conda activate agentic
cd ~/nexaai/project
```

Step 3 — Test schema without submitting (no penalty)

```
python3 main.py --validate-only

# Fix all errors before doing a real submit
# Errors are grouped by category with hints
```

Step 4 — Real submission

```
python3 main.py

# This writes to submissions/ folder
# Instructor collects and scores it
# Scores appear on scoreboard within ~5 minutes
```

Step 5 — Watch the scoreboard

Scoreboard auto-refreshes every 6 seconds. After each round you will see exactly which dimension scored low. Fix that dimension before your next submission.

Deployment Penalty — Read Before Submitting

1st submission: no penalty
2nd submission: -5% on final score
3rd submission: -12%
4th submission: -22%
5th+: -35%

Run --validate-only until schema passes. Then submit once with confidence.

4. Wiring main.py — The Pattern

This is the execution order in main.py. CEO team implements run_department_agents() and run_ceo_agent(). See the comments in main.py for the full template.

```
# REQUIRED EXECUTION ORDER

# 1. HR first — no dependencies
hr = HRAgent()
hr.analyze(scenario)
hr_out = hr.get_output()

# 2. Engineering — needs HR's FTE output
eng = EngineeringAgent()
eng.set_cross_dept_input('hr_effective_fte',
    hr_out['tool_outputs']['model_hiring_pipeline']['effective_fte_at_target_date'])
eng.analyze(scenario)
eng_out = eng.get_output()

# 3. Finance — needs HR + Engineering outputs
fin = FinanceAgent()
fin.set_cross_dept_input('hr_effective_fte', ...)
fin.set_cross_dept_input('engineering_feasibility_prob',
    eng_out['tool_outputs']['estimate_feasibility']['feasibility_probability'])
fin.analyze(scenario)
fin_out = fin.get_output()

# 4. Legal and Marketing — independent
leg = LegalAgent(); leg.analyze(scenario); leg_out = leg.get_output()
mkt = MarketingAgent(); mkt.analyze(scenario); mkt_out = mkt.get_output()
```

5. How Departments Test Their Agents

Each department person tests their own agent independently — they do NOT run main.py. They add a test block at the bottom of their agent file:

```
# At the bottom of agents/finance_agent.py
if __name__ == '__main__':
    agent = FinanceAgent()
    agent.analyze('Test scenario')
    output = agent.get_output()
    import json
    print(json.dumps(output, indent=2, default=str))
```

Then they run: `python3 agents/finance_agent.py`

When you (CEO) are ready to test the full system: `python3 main.py --validate-only`

6. Scoring — What CEO Components Are Worth

Dimension	Weight	What It Tests	Your Component
Incommensurability Resolution	18%	Does CEO converter produce deterministic comparable values from 5 incommensurable inputs?	ValueConverter
Contradiction Detection	18%	Does system detect when new scenario violates prior decisions?	CommitmentLedger
Consensus Stability	18%	Same inputs → same output 3 runs in a row?	ConsensusEngine
Calibration Tracking	12%	Are dept weights updated based on prediction accuracy after Round 1?	CalibrationTracker
Information Gap Detection	12%	Does CEO reduce confidence when depts signal incomplete info?	ConsensusEngine
Hard Constraint Enforcement	10%	Are dept vetoes respected and propagated to CEO?	main.py wiring
Cross-Dept Consistency	7%	Did Engineering use HR's FTE? Did Finance use HR's hiring cost?	main.py data flow
Decision Traceability	5%	Does CEO reasoning cite tool names and values explicitly?	ConsensusEngine

7. Quick Reference

What	URL / Command
Your code editor	http://[YOUR_MACHINE_IP]:8080 (or :8090 for VeriHire AI)
Live scoreboard	http://172.18.40.114:8080/
RAG server	http://172.18.40.114:3000/status
Test schema	python3 main.py --validate-only
Submit	python3 main.py
Project folder	~/nexaai/project/